

Сравнение производительности LAGraph на CPU и spla на GPU

Вячеслав Кочергин, 23.Б11-мм

Постановка задачи

Цель. Сравнить время работы штатных реализаций BFS, SSSP, подсчёта треугольников и PageRank в LAGraph на CPU и в spla на GPU на неориентированных графах различной топологии и объяснить наблюдаемые различия.

Исследовательский вопрос

При каких свойствах алгоритма и графа параллелизм GPU компенсирует запуски ядер OpenCL, синхронизацию с CPU и нерегулярный доступ к памяти?

Объект сравнения

LAGraph / CPU

- BFS: push/pull, эвристика Beamer, вектор родителей
- SSSP: delta-stepping, $\delta = 1$
- TC: SandiaDot, без пресортировки
- PageRank: GAP, останов по L_1

spla / GPU

- BFS: push/pull по плотности фронта, вектор уровней
- SSSP: релаксация изменившихся вершин
- TC: маскированное LL^T ; один work item на ребро маски
- PageRank: останов по L_2

Сравниваются именно готовые реализации по умолчанию, исходные коды примеров не были изменены.

Методика измерения

- Использована `spla-bench`: адаптирована под Python 3.10, унифицирован прогрев, подключены `perf` и `flamegraph`
- `spla` запускается только на GPU: `--run-cpu=false --run-ref=false --run-gpu=true`
- Первый запуск — прогрев; в статистику времени он не входит
- Основная оценка — медиана; отношение $R = T_{\text{LAGraph}} / T_{\text{spla}}$: $R > 1$ — быстрее `spla`, $R < 1$ — быстрее `LAGraph`
- Для крупных графов — 3 измерения, для остальных — 5
- Источник BFS и SSSP: наибольшая компонента связности, степень ближайшая к медиане
- Время чтения Matrix Market не входит в время алгоритма

Экспериментальный стенд

Аппаратное обеспечение	
CPU	Intel Core i5-1135G7 2.40 ГГц, 4 ядра / 8 потоков
RAM	40 ГБ DDR4
L1d	192 КиБ, 4 экземпляра
L1i	128 КиБ, 4 экземпляра
L2	5 МиБ, 4 экземпляра
L3	8 МиБ, 1 экземпляр
GPU	Intel Iris Xe Graphics TigerLake-LP GT2 частота 400 МГц

Программное обеспечение	
OS	Ubuntu 22.04.5 LTS
Ядро	6.8.0-138-generic
perf	6.8.12
Python	3.10.12
CMake	4.4.2
GCC	11.4.0, -03
GraphBLAS	6.1.4, C API 2.0
LAGraph	aa150b1a
spla	b81d1e29
OpenMP	4.5
OpenCL	3.0

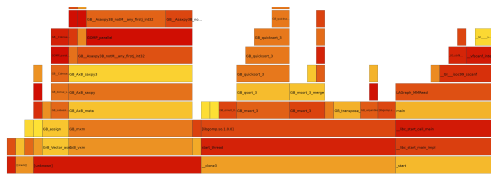
Перед измерениями скрипт подготовки стенда останавливает `irqbalance` и `thermald`, отключает подкачку, через `pyperf` фиксирует частоты CPU без Turbo Boost и ограничивает сэмплирование `perf` до 1 Гц, ослабляет ограничения на аппаратные счётчики и `NMI watchdog`, фиксирует частоту GPU и сбрасывает `page cache`. В выводе `LAGraph` зафиксировано 8 потоков.

Наборы данных

Граф	Вершины	Рёбра	Класс и роль в эксперименте
coAuthorsCiteseer	227 320	814 134	соавторство, умеренная неоднородность
coPapersDBLP	540 486	15 245 729	соавторство статей, вершины высокой степени
belgium_osm	1 441 295	1 549 970	дорожный, большой диаметр, узкий фронт
roadNet-CA	1 971 281	2 766 607	дорожный, узкий фронт
com-Orkut	3 072 441	117 185 083	социальный, степень до 33 313
rgg_n_2_22_s0	4 194 304	30 359 198	случайный геометрический, почти регулярный
rgg_n_2_23_s0	8 388 608	63 501 393	тот же класс, больший масштаб
road_central	14 081 816	16 933 413	крупнейший дорожный граф

- Все восемь графов — неориентированные
- Matrix Market хранит одну треугольную часть; обе библиотеки восстанавливают полную симметричную матрицу
- Набор покрывает узкие длинные обходы, широкие социальные фронты, однородные геометрические графы и экстремальную неоднородность степеней

BFS: преимущество GPU зависит от ширины фронта



Flamegraph LAGraph, road_central1.

Граф	CPU	GPU	R	IPC	LLC
	MC	MC			%
coAuthors	76.0	28.1	2.70	0.47	38.1
coPapers	149.9	94.2	1.59	1.34	49.2
belgium	393.9	1006.1	0.39	0.85	34.6
roadNet	616.9	648.1	0.95	1.06	29.3
Orkut	284.1	232.6	1.22	1.37	38.7
RGG22	4001.8	3062.4	1.31	1.23	44.2
RGG23	8683.6	5623.2	1.54	1.14	43.8
road-central	5614.6	4795.3	1.17	1.11	32.9
медиана	505.4	827.1	1.27	1.13	38.4

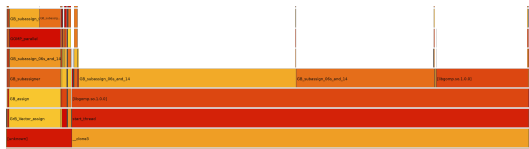
LLC — промахи последнего уровня кэша.

Обоснование

- Главная операция LAGraph BFS — `GB_AxB_saxpy3`: умножение матрицы смежности на вектор фронта
- `perf`: IPC 0,47–1,37 (минимум на `coAuthorsCiteSeer`), промахи кэша 34–49%, LLC 29–49%. CPU ограничен памятью
- На `belgium_osm` фронт остаётся узким. У `spla` каждый уровень — сортировка, редукция и `CLCounter::get` на CPU; малые запуски не окупаются

Вывод. `spla` быстрее на 6 графах из 8, максимум в 2,70 раза. На наиболее узких дорожных обходах быстрее LAGraph.

SSSP: стоимость создаёт управление корзинами



Flamegraph LAGraph, road_central.

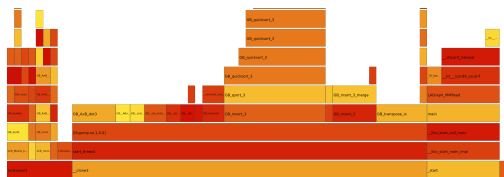
Граф	CPU	GPU	R	IPC	LLC
	MC	MC			%
coAuthors	192.7	64.8	2.98	0.68	29.4
coPapers	836.8	313.6	2.67	1.09	45.9
belgium	40 962	2209	18.54	0.29	27.1
roadNet	19 267	1292	14.92	0.29	36.9
Orkut	2201	1588	1.39	1.28	33.2
RGG22	67 987	8624	7.88	0.52	53.6
RGG23	53 461	16 006	3.34	1.36	54.1
road-central	127 408	9130	13.96	1.80	45.8
медиана	30 115	1899	5.61	0.89	41.3

Обоснование

- Главная операция LAGraph SSSP — GB_subassign_06s_and_14 и GB_bitmap_assign: обновление корзин и глобальных структур
- road_central: $6,6 \cdot 10^{12}$ инструкций, IPC $\approx 1,80$, промахи ветвлений 0,066%, LLC 45,8%
- Ограничение — объём обращений к памяти. При $\delta = 1$ много почти пустых корзин всё равно обновляют глобальные структуры

Вывод. spla быстрее на всех 8 графах. Разрыв 14–19 раз на дорожных сетях создаёт стоимость корзин LAGraph.

Подсчёт треугольников: независимые пересечения выгодны GPU



Flamegraph LAGraph, road_central.

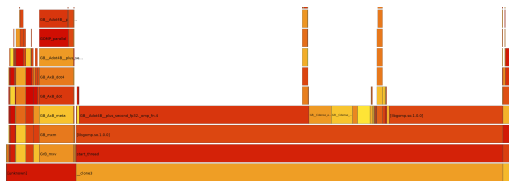
Граф	CPU	GPU	R	IPC	LLC
	MC	MC			%
coAuthors	87.4	14.8	5.89	0.82	29.8
coPapers	1368.0	951.6	1.44	1.60	55.9
belgium	49.2	29.7	1.66	1.61	51.5
roadNet	95.3	41.1	2.32	1.43	58.3
Orkut	51 609	> 3600 c	< 0.014	0.84	58.3
RGG22	1014.7	417.5	2.43	1.38	61.7
RGG23	2165.5	878.6	2.46	1.39	63.7
road-central	723.1	252.5	2.86	1.31	58.5
медиана	868.9	252.5	2.43	1.39	58.3

Обоснование

- Главная операция LAGraph TC — $GB_AxB_dot3 / GB_Adot3B_plus_pair_int64$: маскированное пересечение строк
- У *spla* один work item — одно ребро маски. Пока степени сопоставимы, независимые пересечения выгодны GPU
- На *coPapersDBLP* *R* падает до 1,44. На *com-Orkut* *spla* не завершилась за 3600 с при 51,6 с у LAGraph

Вывод. *spla* быстрее на 7 графах в 1,44–5,89 раза. На *com-Orkut* быстрее LAGraph: $R < 0,014$, не менее чем в 70 раз.

PageRank: регулярный SpMV выгоден GPU



Flamegraph LAGraph, road_central.

Граф	CPU	GPU	R	IPC	LLC
	MC	MC			%
coAuthors	321.8	7.7	41.53	0.30	5.9
coPapers	1194.3	33.1	36.09	0.85	5.8
belgium	1201.2	48.4	24.82	0.31	36.1
roadNet	1317.6	80.7	16.32	0.40	41.5
Orkut	9062.5	545.6	16.61	0.73	23.3
RGG22	1627.0	160.3	10.15	1.03	59.2
RGG23	2737.4	410.2	6.67	1.18	60.0
road-central	5179.0	623.5	8.31	0.67	36.4
медиана	1472	120.5	16.47	0.70	36.3

Обоснование

- Главная операция LAGraph PageRank — GB_AxB_dot4: умножение матрицы смежности на плотный вектор рангов
- perf: наименьший медианный IPC 0,70
- Регулярный SpMV хорошо загружает GPU. Снижение R до 6,67–10,15 на крупных графах согласуется с частотой GPU 400 МГц

Вывод. spla быстрее на всех 8 графах в 6,67–41,53 раза. Часть разрыва методическая: разные нормы, разное число итераций и разное положение нормализации относительно таймера.

- ❶ **BFS.** spla быстрее на 6 графах из 8. perf и flamegraph показывают умножение, ограниченное памятью; проигрыш GPU на дорогах — узкий фронт и синхронизация уровня
- ❷ **SSSP.** spla быстрее на всех графах, максимум в 18,54 раза. Flamegraph относит стоимость LAGraph к assign/subassign, perf — к объёму работы, а не к ветвлениям
- ❸ **ТС.** spla быстрее в 1,44–5,89 раза на 7 графах. На com-Orkut LAGraph считает за 51,6 с, spla не укладывается в час: схема «одно ребро — один work item» не выдерживает неоднородность степеней
- ❹ **PageRank.** spla быстрее в 6,67–41,53 раза. Регулярный SpMV подтверждается IPC 0,70 и операцией GB_AxB_dot4; часть разрыва методическая

Главный вывод

Из 32 сравнений spla/GPU быстрее в 29 случаях; LAGraph выигрывает BFS на узких дорогах и ТС на com-Orkut. GPU выигрывает, когда реализация формирует достаточно крупные независимые работы. При узких фронтах, частых глобальных шагах и сильном дисбалансе степеней алгоритмические накладные расходы важнее пиковой производительности устройства.